

LAPORAN PRAKTIKUM

Thread & Task Parallelism

Laporan ini untuk memenuhi salah satu tugas Mata Kuliah

Komputasi Paralel dan Terdistribusi

Dosen Pengampu: Ir. Randi Rizal, Ph.D.



Disusun Oleh:

Mochammad Fasha Ajvikri (247006111178)

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS SILIWANGI**

2026



1. Tugas Praktikum 1 (I/O-BOUND DOWNLOAD SIMULATOR)

1.1 Tujuan

Memahami penggunaan thread untuk aplikasi I/O-bound serta membandingkan waktu eksekusi serial vs threaded.

1.2 Spesifikasi

Mensimulasikan unduh 10 file dengan durasi acak (0.5–2 detik) secara serial dan threaded, lalu menghitung *speedup*.

1.3 Kode Program

```
# Nama : Mochammad Fasha Ajvikri
# NPM : 247006111178
# Kelas: F

import random
import threading
import time

def download_file(i, sec):
    print(f"Downloading file {i} ({sec:.2f}s)...")
    time.sleep(sec)

def run_serial(jobs):
    print("\nRunning serial download...")
    start = time.time()
    for i, sec in jobs:
        download_file(i, sec)
    return time.time() - start

def run_threads(jobs):
    print("\nRunning threaded download...")
    start = time.time()
    threads = []
    for i, sec in jobs:
        t = threading.Thread(target=download_file,
                              args=(i, sec))
```

```

        threads.append(t)
        t.start()
    for t in threads:
        t.join()
    return time.time() - start

if __name__ == "__main__":
    random.seed(42)
    jobs = [(i, random.uniform(0.5, 2.0)) for i in
range(1, 11)]

    t_serial = run_serial(jobs)
    t_threaded = run_threads(jobs)

    print(
        f"\nSerial Time: {t_serial:.4f}s | Threaded
Time: {t_threaded:.4f}s | "
        f"Speedup: {t_serial/t_threaded:.2f}x"
    )

```

1.4 Output

```

Running serial download...
Downloading file 1 (1.46s)...
Downloading file 2 (0.54s)...
Downloading file 3 (0.91s)...
Downloading file 4 (0.83s)...
Downloading file 5 (1.60s)...
Downloading file 6 (1.52s)...
Downloading file 7 (1.84s)...
Downloading file 8 (0.63s)...
Downloading file 9 (1.13s)...
Downloading file 10 (0.54s)...

Running threaded download...
Downloading file 1 (1.46s)...
Downloading file 2 (0.54s)...
Downloading file 3 (0.91s)...
Downloading file 4 (0.83s)...
Downloading file 5 (1.60s)...
Downloading file 6 (1.52s)...
Downloading file 7 (1.84s)...
Downloading file 8 (0.63s)...
Downloading file 9 (1.13s)...
Downloading file 10 (0.54s)...

Serial Time: 11.0139s | Threaded Time: 1.8403s | Speedup: 5.98x
PS D:\KULIAH\SMT 5\Komputasi Paralel dan Terdistribusi>

```

1.5 Tabel Hasil

Jenis	Jumlah File	Waktu	Speedup
Serial	10	11.0139 s	1.00x
Threaded	10	1.8403 s	6.48x

Pertanyaan Diskusi

- **Mengapa threads lebih efektif untuk aplikasi I/O dibanding serial?**

Aplikasi I/O-bound didominasi oleh waktu tunggu (*idle/waiting state*). Saat satu thread tertahan menanti proses I/O selesai (misal *time.sleep* / koneksi jaringan), CPU langsung beralih (*context switching*) mengeksekusi thread lain. Hal ini membuat banyak pekerjaan berjalan secara bersamaan (konkuren) dan memangkas waktu total secara signifikan.

- **Apa potensi masalah jika terlalu banyak threads dibuat?**
 - a. **Overhead Memory & CPU:** Setiap thread memakan alokasi RAM/stack sendiri dan memicu pemborosan resource akibat *context switching* yang terlalu sering.
 - b. **Resource Exhaustion:** Terlalu banyak koneksi bersamaan dapat menyebabkan pembatasan akses (*rate-limiting*) atau *blocking* oleh server tujuan.

2. Tugas Praktikum 2 (CPU-BOUND HEAVY COMPUTATION)

2.1 Tujuan

Memahami penggunaan `ProcessPoolExecutor` untuk aplikasi CPU-bound serta menghitung nilai *speedup* dan efisiensi.

2.2 Spesifikasi

Mengeksekusi fungsi komputasi berat `heavy(n)` dengan 10^6 iterasi menggunakan 1, 2, 4, dan 8 proses.

2.3 Kode Program

```
# Nama : Mochammad Fasha Ajvikri
# NPM : 247006111178
# Kelas: F

from concurrent.futures import ProcessPoolExecutor
import time

def heavy(n, iters=10**6):
    s = 0
    for i in range(iters):
        s += (i * n) % 7
    return s

def run_test(num_workers, data):
    start = time.time()
    if num_workers == 1:
        list(map(heavy, data))
    else:
        with ProcessPoolExecutor(max_workers=num_workers)
as ex:
        list(ex.map(heavy, data))
    return time.time() - start

if __name__ == "__main__":
    data = list(range(1, 9))
    workers = [1, 2, 4, 8]

    print("\t===ProcessPoolExecutor===\t")
    t_1 = run_test(1, data)
```

```

for w in workers:
    t = run_test(w, data)
    speedup = t_1 / t
    efisiensi = speedup / w
    print(
        f"Proses: {w:2d} | Waktu: {t:.4f}s | Speedup:
{speedup:.2f}x |"
        f" Efisiensi: {efisiensi:.2f}"
    )

```

2.4 Output

```

●      ===ProcessPoolExecutor===
Proses:  1 | Waktu: 0.3692s | Speedup: 1.06x | Efisiensi: 1.06
Proses:  2 | Waktu: 0.3590s | Speedup: 1.09x | Efisiensi: 0.55
Proses:  4 | Waktu: 0.2787s | Speedup: 1.40x | Efisiensi: 0.35
Proses:  8 | Waktu: 0.3731s | Speedup: 1.05x | Efisiensi: 0.13
○ PS D:\KULIAH\SMT 5\Komputasi Paralel dan Terdistribusi>

```

2.5 Tabel Hasil

Proses	Waktu	Speedup	Efisiensi
1	0.3692	1.06x	1.06
2	0.3590	1.09x	0.55
4	0.2787	1.40x	0.35
8	0.3731	1.05x	0.13

Pertanyaan Diskusi

- **Mengapa task parallelism lebih efektif untuk CPU-bound dibanding thread?**

Task parallelism berbasis *Process* memanfaatkan alokasi memori dan ruang eksekusi independen yang berjalan sungguh-sungguh secara paralel di beberapa *core* CPU fisik berbeda, sehingga pemrosesan komputasi murni berjalan maksimal.

- **Apa peran GIL (Global Interpreter Lock) dalam Python?**

GIL merupakan pengunci tingkat eksekusi pada CPython yang membatasi jalannya bytecode Python agar hanya dapat diproses oleh satu thread pada satu waktu. Hal ini menghambat eksekusi multi-threading biasa untuk memanfaatkan multi-core pada beban kerja CPU-bound.

3. Tugas Praktikum 3 (Perbandingan Threads vs Tasks)

3.1 Tujuan

Membandingkan performa *threads* dan *processes* untuk tipe beban kerja I/O-bound dan CPU-bound.

3.2 Spesifikasi

Menguji fungsi dari Tugas 1 & 2 dalam mode Serial, Threads, dan Processes serta menyusun analisis perbandingannya.

3.3 Kode Program

```
# Nama : Mochammad Fasha Ajvikri
# NPM : 247006111178
# Kelas: F

from concurrent.futures import ProcessPoolExecutor,
ThreadPoolExecutor
import random
import time

def io_task(args):
    i, sec = args
    time.sleep(sec)

def cpu_task(n):
    s = 0
    for i in range(10**6):
        s += (i * n) % 7
    return s

if __name__ == "__main__":
    random.seed(42)
    io_jobs = [(i, random.uniform(0.5, 1.5)) for i in
range(8)]
    cpu_jobs = list(range(1, 9))

    # Testing I/O-bound
    st = time.time()
    for j in io_jobs:
        io_task(j)
```

```

t_io_s = time.time() - st

st = time.time()
with ThreadPoolExecutor(max_workers=4) as ex:
    list(ex.map(io_task, io_jobs))
t_io_t = time.time() - st

st = time.time()
with ProcessPoolExecutor(max_workers=4) as ex:
    list(ex.map(io_task, io_jobs))
t_io_p = time.time() - st

# Testing CPU-bound
st = time.time()
list(map(cpu_task, cpu_jobs))
t_cpu_s = time.time() - st

st = time.time()
with ThreadPoolExecutor(max_workers=4) as ex:
    list(ex.map(cpu_task, cpu_jobs))
t_cpu_t = time.time() - st

st = time.time()
with ProcessPoolExecutor(max_workers=4) as ex:
    list(ex.map(cpu_task, cpu_jobs))
t_cpu_p = time.time() - st

print("\n=== HASIL TUGAS 3 ===")
print(
    f"I/O -> Serial: {t_io_s:.2f}s | Threads: {t_io_t:.2f}s | Processes: "
    f"{t_io_p:.2f}s"
)
print(
    f"CPU -> Serial: {t_cpu_s:.2f}s | Threads: {t_cpu_t:.2f}s | Processes: "
    f"{t_cpu_p:.2f}s"
)

```

3.4 Output

```
=== HASIL TUGAS 3 ===  
I/O -> Serial: 7.56s | Threads: 2.20s | Processes: 2.57s  
CPU -> Serial: 1.00s | Threads: 0.94s | Processes: 0.64s  
PS D:\KULIAH\SMT 5\Komputasi Paralel dan Terdistribusi>
```

3.5 Tabel Hasil

Jenis	Serial	Threads	Processes	Speedup Threads	Speedup Processes
I/O	7.56s	2.20s	2.57s	3.95x	3.17x
CPU	1,00s	0.94s	0.64s	0.98x	3.52x

Pertanyaan Diskusi

- **Jelaskan perbedaan hasil antara aplikasi I/O-bound dan CPU-bound!**
 - a. **I/O-bound:** Threads mencatatkan efisiensi waktu terbaik karena beroperasi sangat ringan (*low overhead*). Mode Processes juga menunjukkan hasil cepat, namun terdapat sedikit *delay* pemuatan awal (*spawning process*).
 - b. **CPU-bound:** Threads tidak memberikan percepatan waktu yang berarti akibat terhalang GIL. Sebaliknya, Processes mencatatkan peningkatan kinerja (*speedup*) secara signifikan karena memanfaatkan *core* pemrosesan independen.
- **Apa implikasi pemilihan threads vs processes pada desain sistem nyata?**
 - a. **Gunakan Threads:** Untuk menangani tugas bertipe *I/O-bound* seperti koneksi API, pengambilan data basis data, *web scraping*, dan pengoperasian berkas.

- b. **Gunakan Processes:** Untuk menangani pemrosesan data *CPU-bound* seperti *image processing*, kompresi berkas, *machine learning*, dan perhitungan saintifik.

4. Tugas Praktikum 4 (HYBRID PIPELINE)

4.1 Tujuan

Mendesain pipeline multi-tahap yang memadukan *threads* (I/O loader) dan *processes* (CPU compute) serta mengukur nilai *throughput* dan latensi.

4.2 Spesifikasi

Stage 1 (Thread loader membaca path berkas ke queue), Stage 2 (Process pool mengolah isi berkas), dan Stage 3 (Agregasi hasil).

4.3 Kode Program

```
# Nama : Mochammad Fasha Ajvikri
# NPM : 247006111178
# Kelas: F

from concurrent.futures import ProcessPoolExecutor
import queue # Queue digunakan untuk mentransfer
data antar Thread dan Process secara aman
import threading
import time

def cpu_heavy_processing(data):
    s = 0
    for i in range(10**6): # Simulasi kalkulasi
matematika berat (1 juta kali loop)
        s += (i * data) % 7
    return s

def stage1_loader(file_paths, q):
    for path in file_paths:
        time.sleep(0.05) # Simulasi jeda I/O (membaca
file dari storage)

        q.put((path, len(path)))

    q.put(None)

def run_pipeline(files, num_processes):
```

```

    start_time = time.time()    # Catat waktu awal
pipeline dimulai

    # maxsize=5 membatasi antrean di memori agar tidak
terjadi Out of Memory (OOM)
    q = queue.Queue(maxsize=5)

    # --- STAGE 1: Jalankan Thread Loader ---
    # Thread ini bertugas membaca file di background
tanpa menghentikan alur utama
    t_loader = threading.Thread(target=stage1_loader,
args=(files, q))
    t_loader.start()

    results = []

    # --- STAGE 2: Process Pool untuk Komputasi CPU ---
    # Menggunakan multi-process agar perhitungan berat
memanfaatkan multi-core CPU
    with ProcessPoolExecutor(max_workers=num_processes)
as executor:
        futures = []

        # Loop mengambil item dari Queue secara
terus-menerus
        while True:
            item = q.get()    # Mengambil item terdepan dari
antrean

            if item is None:    # Jika mengambil None
(Sentinel), artinya Stage 1 selesai
                break

            # Lempar tugas komputasi berat ke salah satu
Process Worker yang bebas
            f = executor.submit(cpu_heavy_processing,
item[1])
            futures.append(f)

    # --- STAGE 3: Agregasi Hasil ---

```

```

        # Mengumpulkan semua hasil komputasi dari Process
Pool
        for f in futures:
            results.append(f.result())

        # Pastikan Thread Loader sudah selesai sepenuhnya
        sebelum lanjut
        t_loader.join()

        # --- KALKULASI METRIK PERFORMA ---
        total_time = time.time() - start_time # Total
waktu dari awal hingga selesai
        throughput = len(files) / total_time # Jumlah file
yang terproses per detik
        avg_latency = total_time / len(files) # Rata-rata
waktu untuk memproses 1 file

        return total_time, throughput, avg_latency

#
=====
=====
# MAIN BLOCK
#
=====
=====
if __name__ == "__main__":
    # Siapkan dummy dataset berisi 20 file
    files = [f"file_{i}.txt" for i in range(20)]

    print("\n" + "=" * 60)
    print(">>>  MEMULAI  SIMULASI  HYBRID  PIPELINE
(THREADS + PROCESSES) <<<")
    print("=" * 60)

    # Jalankan pipeline dengan 4 Worker Process
    total_time, tp, lat = run_pipeline(files,
num_processes=4)

    # Cetak Ringkasan Performa

```

```

    print(f"Jumlah File yang Diproses : {len(files)}
file")
    print(f"Jumlah Process Worker CPU : 4 workers")
    print(f"Waktu Total Eksekusi : {total_time:.4f} detik")
    print(f"Throughput : {tp:.2f} file/detik")
    print(f"Rata-rata Latensi : {lat:.4f} detik/file")
    print("=" * 60)

```

4.4 Output

```

=====
>>> MEMULAI SIMULASI HYBRID PIPELINE (THREADS + PROCESSES) <<<
=====
Jumlah File yang Diproses : 20 file
Jumlah Process Worker CPU : 4 workers
Waktu Total Eksekusi      : 1.2519 detik
Throughput                : 15.98 file/detik
Rata-rata Latensi         : 0.0626 detik/file
=====
PS D:\KULIAH\SMT 5\Komputasi Paralel dan Terdistribusi>

```

4.5 Tabel Hasil

Threads Loader	Worker CPU	Jumlah File	Waktu	Throughput	Avg Latency
1	4	20	1.2519s	15.98 file/s	0.0626 s

Pertanyaan Diskusi

- Di mana bottleneck pipeline terjadi ?

Bottleneck terletak pada tahap pemrosesan dengan durasi terlama (*slowest stage*). Pada pengujian ini, titik *bottleneck* umumnya terjadi pada **Stage 2 (CPU Compute)** apabila jumlah *worker process* kurang memadai untuk mengimbangi kecepatan pasokan data dari *thread loader*.

- **Bagaimana peran backpressure (queue maxsize) dalam pipeline ini?**

Parameter maxsize pada Queue berfungsi menahan laju Stage 1 (*loader*) agar berhenti sejenak mengisi antrean jika Stage 2 (*CPU worker*) belum selesai mengolah data sebelumnya. Hal ini mencegah konsumsi RAM melonjak ekstrim (*Out of Memory*).

- **Bagaimana menyesuaikan jumlah thread vs proses untuk optimasi?**

- a. **Jumlah Thread:** Disesuaikan dengan batas kecepatan I/O storage atau *bandwidth* jaringan.
- b. **Jumlah Process:** Disesuaikan dengan jumlah core fisik CPU mesin agar tidak terjadi krisis perebutan *resource* (*core thrashing*).

KESIMPULAN

- a. **Thread Parallelism** sangat optimal untuk aplikasi **I/O-bound** karena tidak terhambat oleh pengunci GIL pada Python serta memakan alokasi RAM yang ringan.
- b. **Task Parallelism (Process)** merupakan solusi mutlak untuk aplikasi **CPU-bound** karena sanggup mengeksekusi komputasi secara paralel murni di multi-core fisik CPU.
- c. Penggabungan keduanya dalam **Hybrid Pipeline** menghasilkan arsitektur sistem ideal: Stage I/O dikelola efisien oleh *thread*, sedangkan Stage kalkulasi berat diproses secara independen oleh *process pool*.